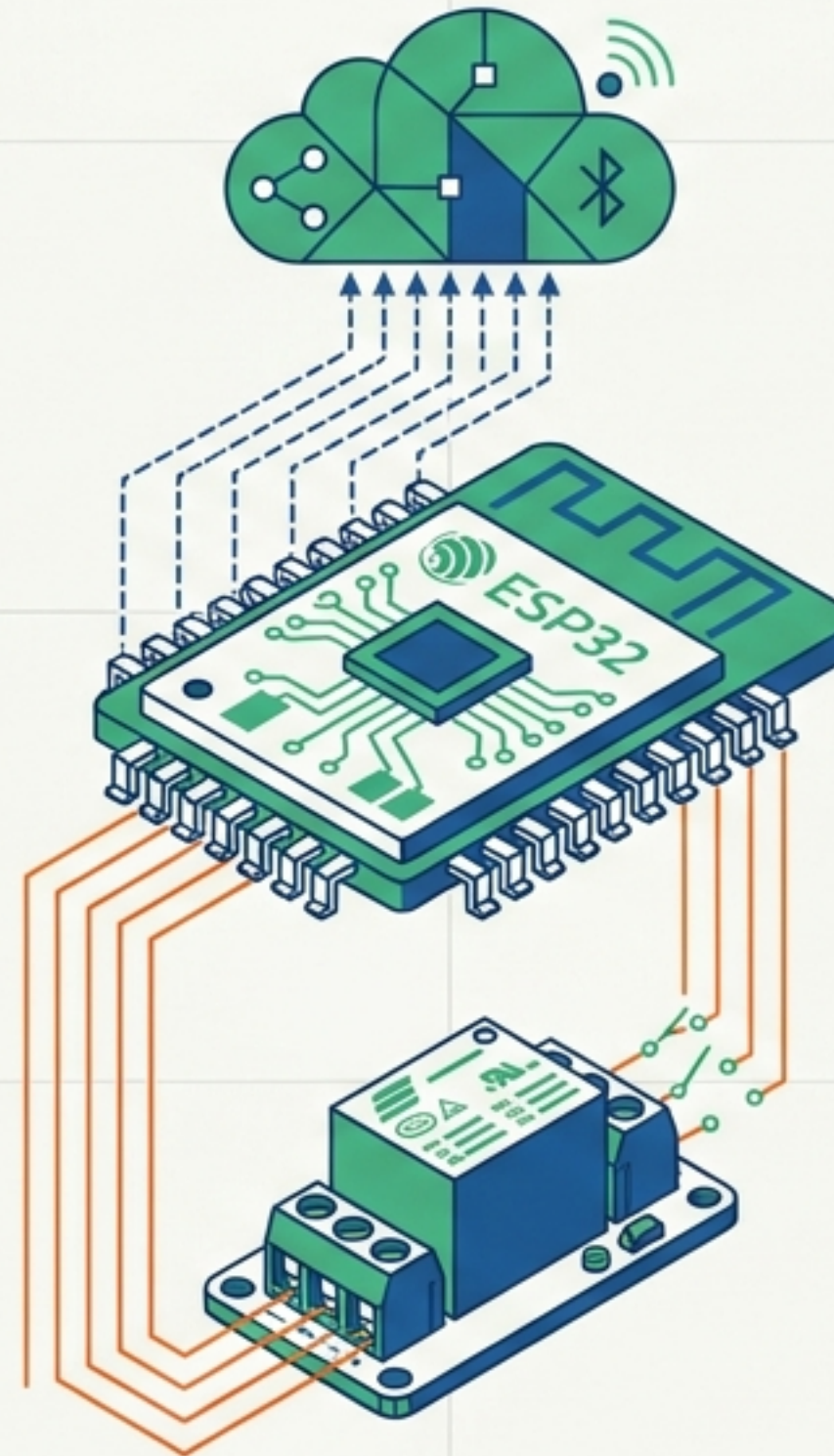


Mastering Blynk IoT

จากการอ่าน Documentation สู่การควบคุมฮาร์ดแวร์จริง



หลักสูตรสอนทำระบบ IoT ด้วย Arduino โดย Ampify

ทำไมถึงไม่สอนแบบ ‘ป้อนโค้ด’ (Spoon-feeding)?

วิธีเดิม (The Old Way)



คัดลอกโค้ดและท่องจำ



ระบบพังเมื่อแพลตฟอร์ม
อัปเดตเวอร์ชันใหม่



ยึดติดกับแพลตฟอร์มเดียว
เปลี่ยนไปใช้ตัวอื่นไม่ได้

วิธีของนักพัฒนา (The Pro Way)



อ่าน Documentation
จากผู้สร้างโดยตรง



ปรับแก้โค้ดได้เองทุกครั้ง
ที่มีการเปลี่ยนแปลง UI

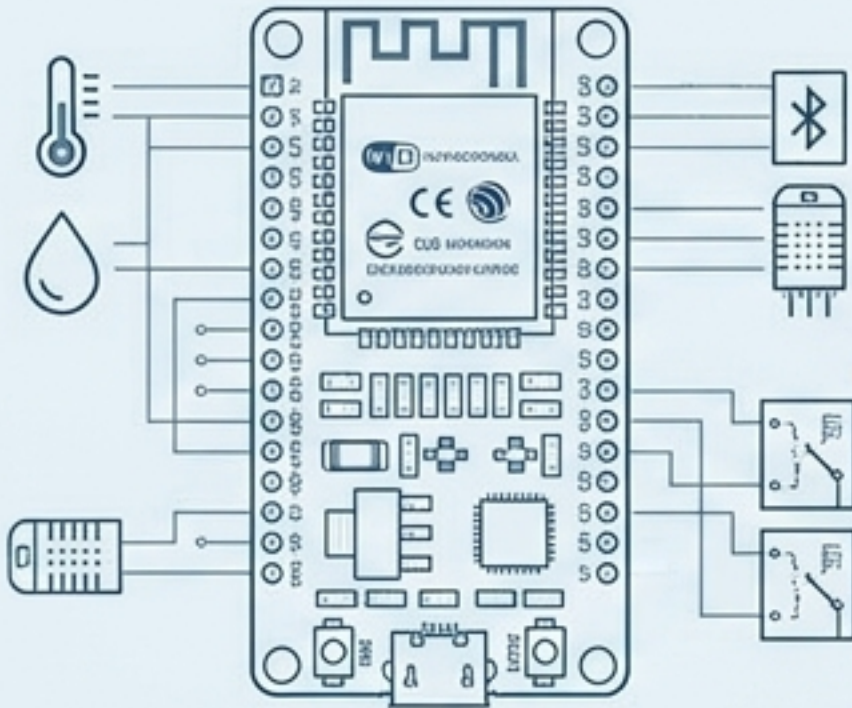


นำทักษะไปประยุกต์ใช้กับ
แพลตฟอร์ม IoT อื่นๆ ได้ทั้งหมด
(สอนให้จับปลา แทนการให้ปลา)

Blynk คืออะไร?

Low-Code IoT Platform ที่เชื่อมต่อฮาร์ดแวร์เข้ากับระบบคลาวด์

ฮาร์ดแวร์



บอร์ดไมโครคอนโทรลเลอร์
(เช่น ESP32)
รับคำสั่งและควบคุมเซนเซอร์/รีเลย์

Blynk Cloud



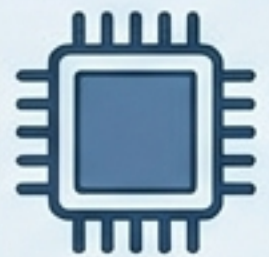
ศูนย์กลางประมวลผลและส่งต่อข้อมูล
(ตัวกลางระหว่างฮาร์ดแวร์และแอป)

ผู้ใช้งาน



ควบคุมและดูสถานะผ่านหน้าเว็บ
(Blynk Console)
หรือแอปพลิเคชันบนสมาร์ทโฟน

สิ่งที่ต้องเตรียมพร้อม (Prerequisites)



อุปกรณ์
(Hardware)

บอร์ด ESP32
(Supported Hardware
สำหรับการเชื่อมต่อ WiFi)



พื้นฐานไฟฟ้า
(Electronics)

เข้าใจวงจรไฟฟ้าพื้นฐานและ
การต่อสายสัญญาณ



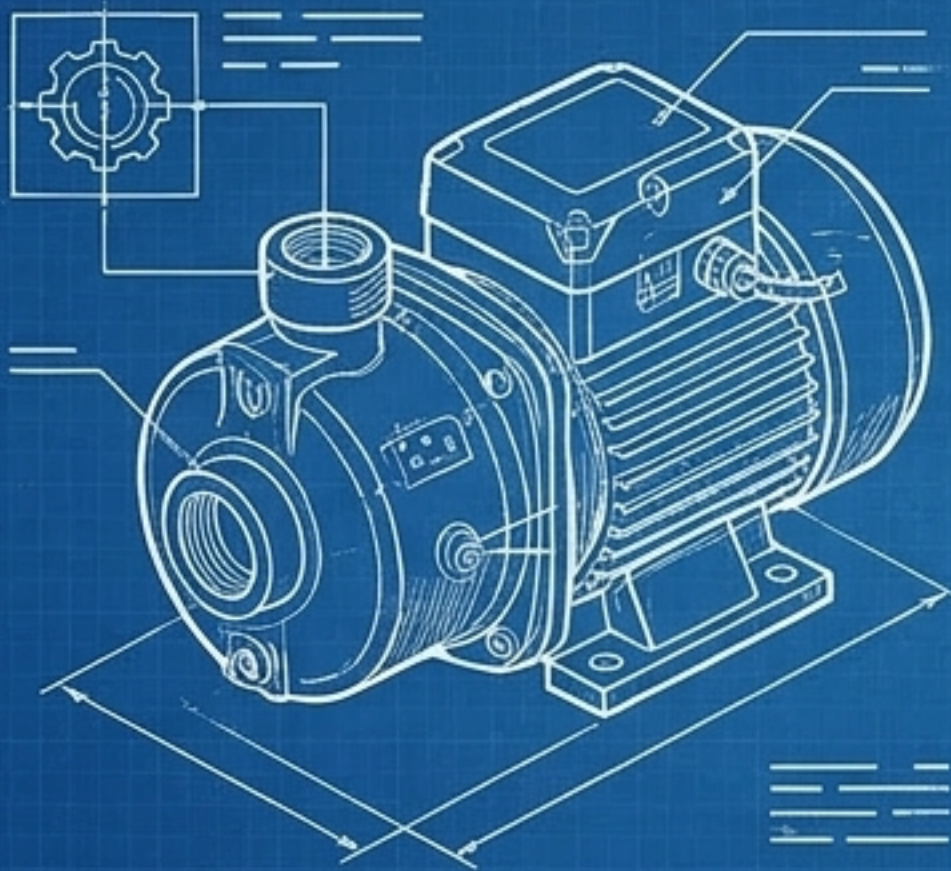
พื้นฐานโค้ด
(Coding)

เขียนโปรแกรมไฟกะพริบ
(Blink LED)
บน Arduino IDE ได้

แนวคิดหลัก: 'Device Template'

การสร้างระบบที่ขยายสเกลได้ ต้องเริ่มต้นจาก 'แม่พิมพ์'

แม่พิมพ์ (Template): ป้อนน้ำอ้อยริยะ



นำไปสร้าง
(Instantiate)

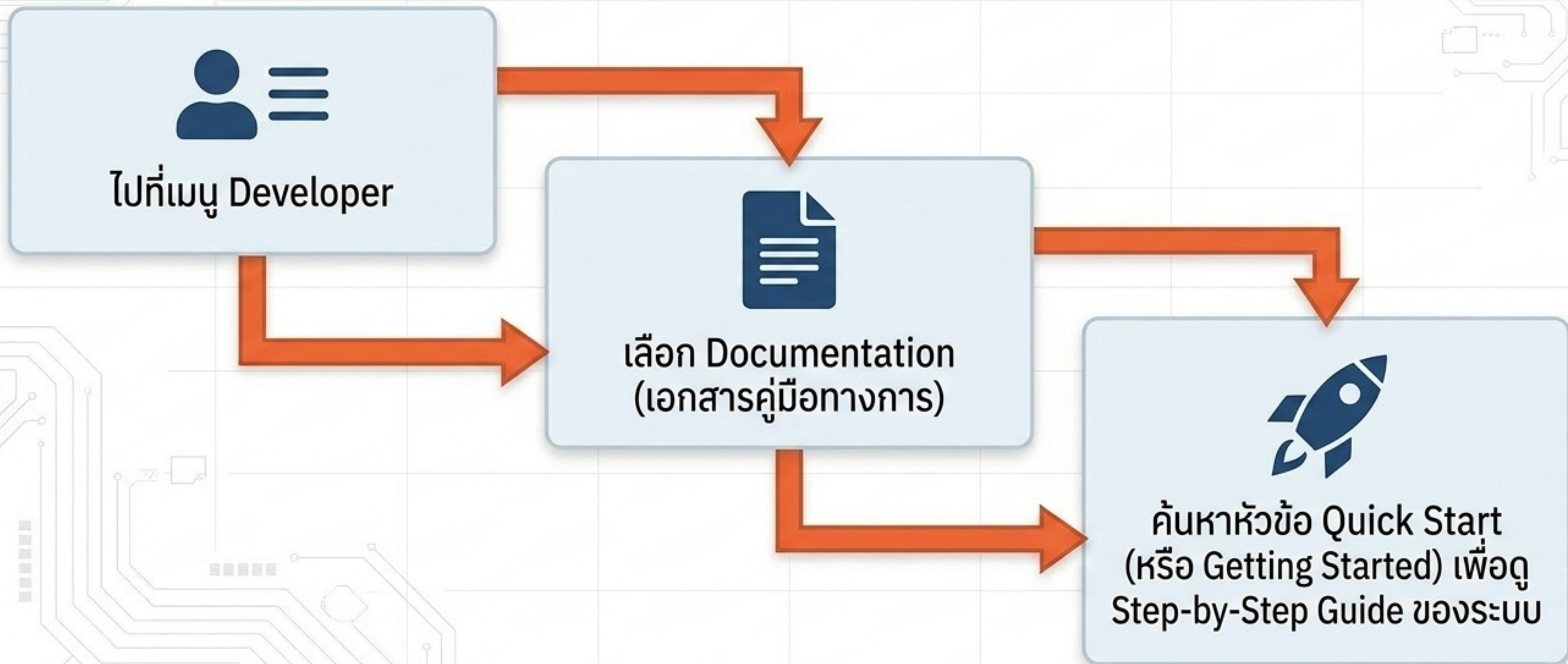


กำหนดค่า Configuration พื้นฐานไว้เพียงครั้งเดียว

อุปกรณ์จริง (Devices) 20 ตัว!
ทุกตัวจะใช้สเปกและการตั้งค่าเดียวกันทั้งหมด
ช่วยลดเวลาจัดการมหาศาล

แผนที่นำทาง: ค้นหา Quick Start

จุดเริ่มต้นของการใช้งานแพลตฟอร์มใหม่คือการหา Documentation



การตั้งค่าบน Blynk Console

ค้นหา 'Quick Start' ในหน้า Devices และตั้งค่าพื้นฐานดังนี้:

Hardware

ESP32

ESP8266

Arduino UNO

Connection

WiFi

Ethernet

Platform

Arduino

PlatformIO

หมายเหตุ: สำหรับผู้ใช้ทั่วไป (โปรเจกต์คนเดียว) โหมด Developer Mode จะถูกเปิดไว้เป็นค่าเริ่มต้น (Default) อยู่แล้ว

การติดตั้ง Blynk Library

เชื่อมต่อโค้ดเข้ากับระบบคลาวด์

🔍 Blynk

Blynk

Volodymyr Shymanskyy

ต้องเป็นของนักพัฒนาชื่อ
Volodymyr Shymanskyy เท่านั้น!

Code Anatomy: การเชื่อมต่อเครือข่าย

คัดลอกโค้ดจากหน้าเว็บมาทับใน Arduino IDE และปรับแก้ข้อมูล WiFi

ใส่ชื่อ WiFi ของคุณ

```
// Your WiFi credentials.  
// Set password to "" for open networks.  
char ssid[] = "YOUR_WIFI_NAME";  
char pass[] = "YOUR_WIFI_PASSWORD";
```

ใส่รหัสผ่าน WiFi (หาก WiFi
ไม่มีรหัสผ่าน ให้เว้นว่างไว้เป็น "")

กฎเหล็กของการเชื่อมต่อ ESP32

การเลือกย่านความถี่ WiFi (WiFi Band)

WiFi 2.4GHz



สถานะ: รองรับการใช้งานเต็มรูปแบบ
(Supported)

เหตุผล: เป็นชิปมาตรฐานของ ESP32
เหมาะสำหรับ IoT และทะลุ
ทะลวงสิ่งกีดขวางได้ดี

WiFi 5GHz



สถานะ: ไม่รองรับ (Not Supported)

ผลลัพธ์: บอร์ดจะไม่สามารถเชื่อมต่อ
อินเทอร์เน็ตได้ ระบบจะไม่ออนไลน์

Code Anatomy: การปรับค่า Timer (Best Practice)

```
void setup() {  
  Serial.begin(115200);  
  Blynk.begin(auth, ssid, pass);  
  timer.setInterval(1000L 600000L, myTimerEvent);  
}
```



เปลี่ยนตัวเลขจาก 1000 → เป็น 600000 (หกแสน)

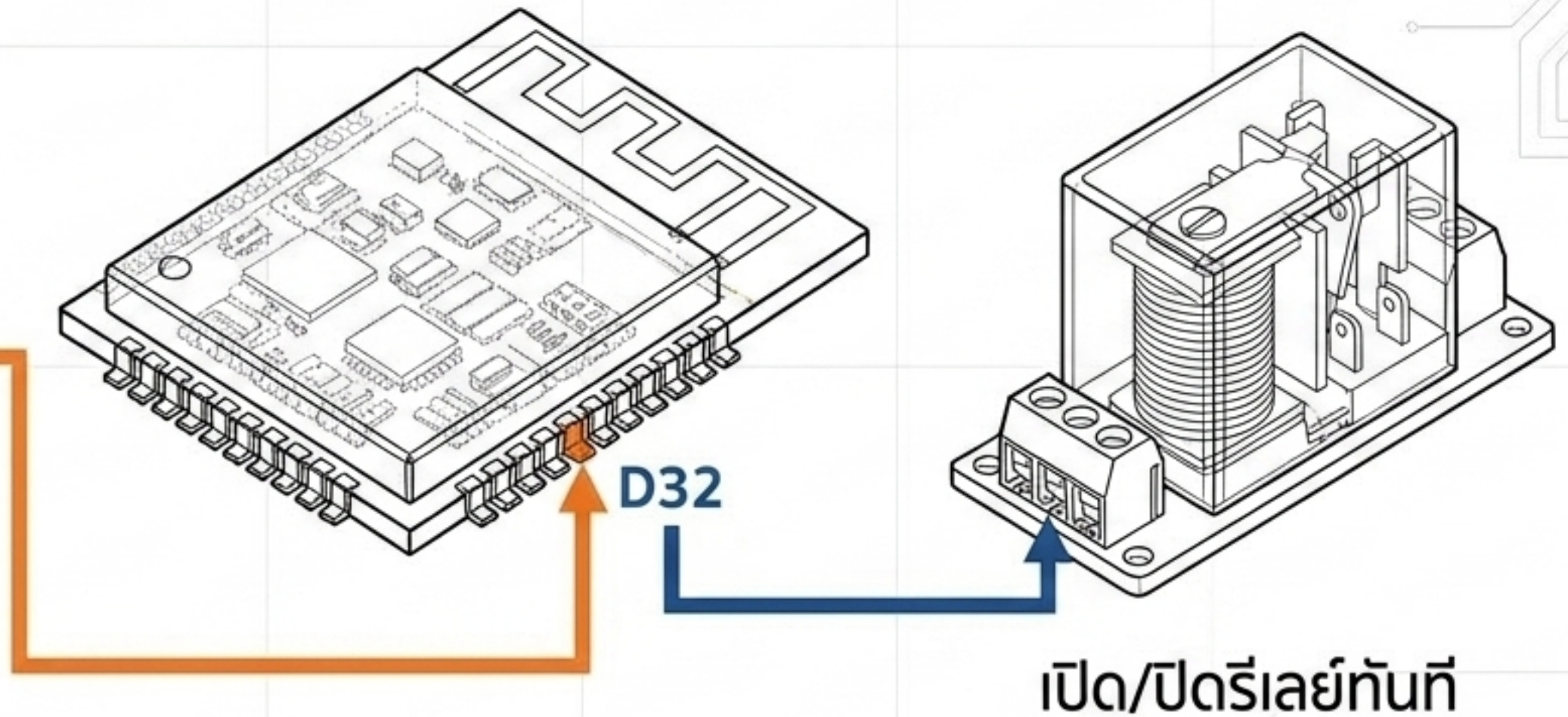
นี่คือจุดที่สำคัญมาก! การเปลี่ยนค่านี้เกี่ยวข้องกับเสถียรภาพของระบบ (รายละเอียดเชิงลึกจะอธิบายในตอนถัดไป)

Hardware Logic: สั่งการรีเลย์

นำคำสั่งจากคลาวด์ส่งตรงไปยังขั้ว Pin ของฮาร์ดแวร์

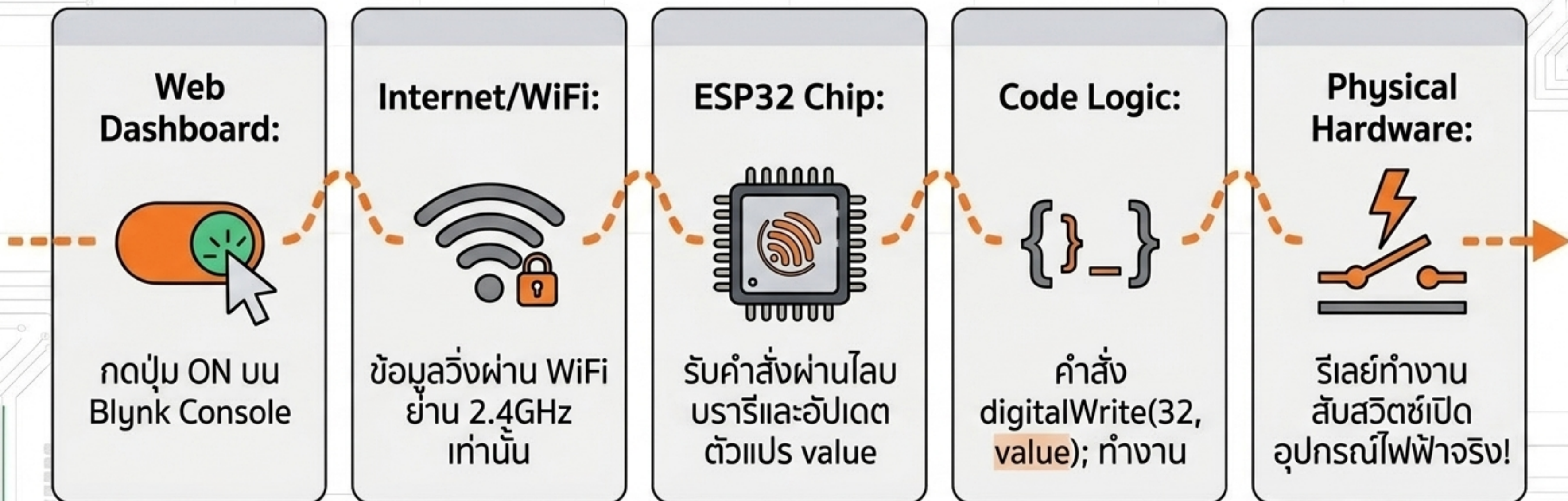
```
pinMode(32, OUTPUT);  
digitalWrite(32, value);
```

รับค่าสถานะ (เปิด/ปิด) จากคลาวด์
ที่ส่งเข้ามาผ่านตัวแปร value
และสั่งเปลี่ยนสถานะขา 32



Full System Synthesis: การทำงานเต็มระบบ

อุปการสั่งงานจากหน้าจอคอมพิวเตอร์ สู่การทำงานของฮาร์ดแวร์



ตรวจสอบสถานะการทำงาน



Your device is Online

สถานะในหน้า Devices ต้องขึ้นว่า 'Online' แปลว่าบอร์ดเชื่อมต่อกลาวด์สำเร็จ!



เช็คชื่อ WiFi และ รหัสผ่าน
พิมพ์ผิดหรือไม่?
(ตัวพิมพ์เล็ก/ใหญ่มีผล)



เปลือยใช้เครือข่าย
5GHz หรือไม่?
(ต้อง 2.4GHz เท่านั้น)

“ ”

หาก WiFi ไม่มีรหัส
ห้ามเว้นช่องว่างเปล่าๆ
ให้พิมพ์สัญลักษณ์ ""
(String ว่าง) ไว้ในโค้ด

พร้อมสร้างโปรเจกต์ต่อไปหรือยัง?



หากคุณกำลังมองหาอุปกรณ์สำหรับงาน IoT และ Arduino สามารถสนับสนุนช่องของเราได้ที่ร้านค้าของ Ampify



ค้นหา 'โลโก้รูปใบไม้ในกรอบสีน้ำเงิน'
บน Shopee, Lazada และ TikTok Shop

ขอบคุณที่ร่วมเดินทางไปกับการเรียนรู้ที่ไม่มีวันสิ้นสุด!